

Конспект по оптимизации запросов к БД в Laravel.

1) Сокращайте использование памяти, разбивая набор записей на части.

```
$posts = Post::chunk(100, function($posts) {  
    foreach ($posts as $post){  
        // ...  
    }  
});
```

2) Не используйте `select *` (выбор всех полей).

Лучше так:

```
// только поля 'id' и 'name'  
$posts = Post::where('name','=', 'vasia')  
    ->first(['id', 'name']);
```

3) Правильно считайте кол-во записей в таблице.

```
// Правильно:  
// select count(*) from posts  
$posts = Post::count();  
  
// Не правильно:  
// ( select * from posts )->count()  
$posts = Post::all()->count();
```

4) Избегайте проблемы `N+1`. Используйте `жадную загрузку`.

Проблема `N+1` - это, когда для каждой записи основной таблицы выполняется отдельный запрос к связанной таблице. Вместо этого можно один запрос к связанной таблице для всех записей из основной таблицы.

```
$posts = Post::with(['user']->get(['** */]));  
  
// Для вложенных отношений  
$posts = Post::with(['user.team']->get(['** */]));
```

5) Используйте индексацию для часто запрашиваемых полей.

```
// Миграция  
Schema::table('posts', function (Blueprint $table) {  
    $table->index('email');  
});
```

6) Предпочитайте использовать `simplePaginate` вместо `Paginate` (`simplePaginate` - не может считать общее кол-во записей).

7) Выделяйте поля с большими данными в отдельную таблицу.

8) Для получения последней записи `id` лучше, чем `created_at` (сортировка по `id` быстрее).

Лучше:

```
$posts = Post::latest('id')->get();  
// или  
$posts = Post::orderBy('id', 'desc')->get();
```

Хуже:

```
$posts = Post::latest()->get();  
// или  
$posts = Post::orderBy('created_at', 'desc')->get();
```

9) Удаляйте не используемые индексы

10) Не используйте индексы на небольших (до нескольких тысяч записей) таблицах

11) Создавайте индексы только под медленные запросы

Поиск медленных запросов

```
// AppServiceProvider:  
public function boot()  
{  
    DB::listen(function ($query) {  
        $stackTrace = collect(debug_backtrace())  
            ->filter(function ($trace) {  
                return !str_contains($trace['file'], 'vendor/');  
            });  
  
        dd($stackTrace);  
    });  
}
```

Логирование:

```
public function boot()
{
    DB::listen(function ($query) {
        $location = collect(debug_backtrace())
            ->filter(function ($trace) {
                return !str_contains($trace['file'], 'vendor/');
            })->first(); // берем первый элемент не из каталога вендора

        $bindings = implode(", ", $query->bindings); // форматируем привязку как строку

        Log::info("
            -----
            Sql: $query->sql
            Bindings: $bindings
            Time: $query->time
            File: ${location['file']}
            Line: ${location['line']}
            -----
        ");
    });
}
```